Step 1 — Two Standalone MCP-Connected Specialists — Hints

> Back to the step · Hint 1 · Hint 2 · Solution

- Hint 1 — The idea, and the exact pieces
- Hint 2 — The plan, and the one shared loop

Hint 1 — The idea, and the exact pieces

Two specialists, each owning its own MCP connection, each proven alone. Reuse what you already built:

- `servers/notes_server.py` + `servers/web_server.py` — tiny `MCPServer`s (Project 7's shape), the web one a mocked URL->text dict.
- `src/mcp_client.py` — `run_specialist(client, command, args, system, task)`: open `stdio_client` / `ClientSession`, `initialize()`, `list_tools()`, run a ReAct loop (Project 8's tool-bridge + loop). Returns `(findings, unavailable)`.
- `src/agents/notes_agent.py`, `web_agent.py` — thin wrappers calling `run_specialist` with their server.
- `main.py` — run both standalone (async) and print.

Hint 2 — The plan, and the one shared loop

```
run_specialist(client, command, args, system, task):
    async with stdio_client(params) as (r,w):
        async with ClientSession(r,w) as session:
            await asyncio.wait_for(session.initialize(), timeout=...)
            tools = list_tools() -> openai schema (name/desc/schema)
            loop: chat.completions.create(tools=...) -> tool_calls?
                  run session.call_tool(...), append tool message, repeat
            return (answer, False)
    except Exception: return ("", True)   # degradation lives here (Step 4)
```

The key point: **the connection is opened and closed inside this one async call**, so when a specialist runs as a graph node later, the enter/exit happen in the same task and teardown never

crosses tasks (the trap Project 8 hit). Prove each agent standalone before Step 2. The whole file set is in the Solution.